



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



تمرین ۲

سوال ۱

(آ)

از ورژن $HTTP\ 1.1$ استفاده می‌کند.

(ب)

در خط اول از روی $status\ code = 200$ با $status\ phrase = OK$ متوجه می‌شویم که سرور توانسته است با موفقیت سند را ارسال کند.

(ج)

اندازه سند در بخش $Content-length$ مشخص شده که برابر 53,993 بایت است. طبق [۱] همیشه فیلد $Content-length$ به بایت گزارش می‌دهد.

(د)

نوع فایل در بخش $Content-type$ مشخص شده که از نوع $image/html$ است.

سوال ۲

در معماری $client-server$ ، تمام فایل‌ها باید توسط خود سرور ارسال شود. پس باید فایل $F = 5Gbits$ به $N = 8$ نفر با نرخ آپلود سرور $u_s = 87Mbps$ ارسال شود.

از طرفی کمینه نرخ دانلود برای همتای اول است و برابرست با $d_{min} = 10Mbps$. پس تاخیر $end\ to\ end$ ارسال در معماری $client-server$ برابرست با:

$$D_{c-s} \geq \max\{NF/u_s, F/d_{min}\} = \max\left\{\frac{8 \times 5 \times 10^9}{87 \times 10^6}, \frac{5 \times 10^9}{10 \times 10^6}\right\} = \frac{5 \times 10^9}{10 \times 10^6} = 500 \text{ secs}$$

در صورتی که در روش *peer-to-peer*، چون از نرخ آپلود هم‌تاها نیز کمک گرفته می‌شد، فرمول کلی آن به این صورت می‌شد:

$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(\sum u_i + u_s)\} = \max\left\{\frac{5 \times 10^9}{87 \times 10^6}, \frac{5 \times 10^9}{10 \times 10^6}, \frac{8 \times 5 \times 10^9}{256 \times 10^6}\right\}$$

$$= \frac{5 \times 10^9}{10 \times 10^6} = 500 \text{ secs}$$

از آنجایی که نرخ دانلود هم‌تای اول *bottleneck* است، در هر دو مورد، همین تاخیر دانلود آن بیشترین تاخیر را ایجاد می‌کند.

سوال ۵

iterative request

* بین مراحل یک و دو، *DNS* محلی در صورتی که از قابلیت *caching* بهره ببرد. ابتدا خودش یعنی *DNS local* را چک می‌کند که آیا کش مربوط به نگاشت اسم به آدرس *IP* را دارد یا خیر. اگر از قابلیت *caching* بهره نمی‌برد و یا کش نگاشت مربوطه را نداشت، آن *query* را به *DNS root* فوروارد می‌کند. * اگر *DNS root*، آدرس *IP* مورد نظر را نداشت، پاسخ را به *DNS TLD* مربوطه ارجاع می‌دهد. (یعنی به *local DNS* پاسخ می‌دهد که آدرس اصلی را نداشته اما آدرس *TLD* آن را دارد و خودت به آن مراجعه کن) * اگر *DNS TLD* نیز آدرس *IP* مورد نظر را نداشت، پاسخ را به *DNS authoritative* ارجاع می‌دهد. (باز مثل قبل، *DNS TLD* به *DNS* محلی پاسخ می‌دهد که من آدرس دقیق را ندارم و خودت باید به آدرس *DNS authoritative* که الان بهت میدم، مراجعه کنی.) * بعد که *DNS authoritative* پاسخ داد، یک رکورد تایپ *A* به *DNS local* بازگردانده می‌شود که همان نگاشت اسم *canonic* به آدرس *IP* سایت مورد نظر است.

recursive request

* برای درخواست های *recursive* هم همان قبلی. بین مراحل یک و دو، *DNS* محلی در صورتی که از قابلیت *caching* بهره ببرد. ابتدا خودش یعنی *DNS local* را چک می‌کند که آیا کش مربوط به نگاشت اسم به آدرس *IP* را دارد یا خیر.

اگر از قابلیت *caching* بهره نمی‌برد و یا کش نگاشت مربوطه را نداشت، آن *query* را به *DNS root* فوروارد می‌کند.

- * سرور *DNS* ریشه، درخواست را به *DNS TLD* فوروارد می‌کند.
- * درواقع سرور معتبر *DNS* یا همان *DNS authoritative* پاسخ را به *DNS TLD* برمیگرداند.
- * رکورد از نوع تایپ *A* بازگردانده می‌شود.
- * مقایسه بین این دو بستگی دارد که از چه نظر بررسی می‌کنیم:

در روش *recursive* می‌توان نگاشت‌ها را در سرورهای وسط راه *cache* کرد و در پرس و جوهای بعدی لازم نیست که کل مسیر را طی کنند. عیب این روش این است که لود زیادی از سرورهای میانی و مخصوصاً سرورهای ریشه قرار می‌گیرد چون به نوعی همه درخواست‌ها از سرور ریشه گذشته و توسط سرور ریشه پاسخ داده می‌شود. در صورتی که در روش *iterative* ریشه تنها آدرس *DNS TLD* مربوطه را میداد و لود بقیه کار روی دوش *local DNS* بود.

سوال ۷

خب طبق اطلاعات داده شده، هنگامی که از کش استفاده نکنیم، متوسط زمان دریافت فایل ۲ ثانیه است و اگر از کش استفاده کنیم، با فرض اینکه ۴۰ درصد *cache hit* داریم و ۶۰ درصد *cache miss*، میانگین زمان دریافت کل فایل‌ها به ۱.۷ ثانیه کاهش میابد. حال باید زمان دریافت فایل داخل *LAN* که از طریق *cache hit* رخ میدهد را حساب کنیم [۲]. پس داریم:

$$T_{avg} = (H \times T_{LAN}) + (M \times T_{net})$$

$$\Rightarrow 1.7 = (0.4 \times T_{LAN}) + (0.6 \times 2) \Rightarrow T_{LAN} = \frac{1.7 - 1.2}{0.4} = 1.25 \text{ sec}$$

سوال ۹

* در مورد اول، اتصال‌ها *non - persistent* و سری هستند. در کل ۶ آبجکت داریم پس داریم:

$$T = \sum_{i=1}^6 (2 \times RTT_i + FTT_i) = 12 \times RTT + \sum_{i=1}^6 FTT_i$$

$$\Rightarrow T = 12 \times 100 \text{ ns} + \frac{(1 + 5 \times 50) \times 8 \times 1024}{10^6} \simeq 1.2 \times 10^{-6} + 2.056192 \text{ sec} = 2.0561932 \text{ sec}$$

* در مورد دوم، اتصال‌ها هنوز *non - persistent* هستند با توانایی داشتن دو اتصال موازی همزمان. پس: ابتدا باید سند را دریافت کنیم. داریم:

$$T_{base} = 2 \times RTT + FTT_{base} = 2 \times 100 \text{ ns} + \frac{1024 \times 8}{10^6} = 2 \times 10^{-7} + 8.192 \times 10^{-3} \text{ sec}$$

بعد ۵ تا عکس را میتوانیم در ۳ دوره دریافت کنیم. دوره اول، تصاویر ۱ و ۲، دوره دوم تصاویر ۳ و ۴ و دوره سوم تصویر ۵. پس در کل داریم:

$$T_{img} = 3 \times (2 \times RTT + FTT_{img}) = 3 \times (200 \text{ ns} + \frac{50 \times 8 \times 1024}{10^6}) = 3 \times 200 \text{ ns} + 3 \times 0.4096$$

$$= 6 \times 10^{-7} + 1.2288 \text{ sec}$$

پس در کل داریم:

$$T_{total} = 8 \times 10^{-7} + 1.2288 + 0.008192 = 1.2369928 \text{ sec}$$

* در مورد سوم، ابتدا سند را در همان T_{base} بخش قبل، بدست میاوریم.

$$T_{base} = 2 \times 10^{-7} + 8.192 \times 10^{-3} \text{ sec}$$

اما چون اتصال بسته نیست، برای آبجکت های بعدی فقط یک RTT مصرف می‌شود. پس داریم:

$$T_{img} = 5 \times (RTT + FTT_{img}) = 5 \times (100 \text{ ns} + \frac{50 \times 8 \times 1024}{10^6}) = 5 \times 10^{-7} + 2.048 \text{ sec}$$

پس در کل داریم:

$$T_{total} = 7 \times 10^{-7} + 2.048 + 0.008192 = 2.0561927 \text{ sec}$$

سوال ۱۰

برای نوشتن کد های این بخش از جمینای [۳] و اسلاید های درس کمک گرفتیم.

کد تکمیل شده را در فایل server.py قرار دادم که در عکس ۱ میتوانید آن را مشاهده کنید.

برای آدرس IP نیز دستورات این عکس ۲ را زدم.

یک فایل HelloWorld.html ساده هم نوشتم که در تصویر ۳ میتواند مشاهده کنید. همانطور که در ۳ میبینید، دو

تا درخواست از بروزر دادم. یکی:

<http://172.17.70.236:6789/HelloWorld.html>

و دیگری:

<http://172.17.70.236:6789/FileNotFound.html>

که درواقع اولی یک درخواست درست است و فایل آن موجود است و در درخواست دوم، فایل مذکور، موجود نیست.

حال می‌توانید چیزی که در بروزر دیدم را در عکس ۴ و ۵ ببینید.

The screenshot shows a code editor with two files: `HelloWorld.html` and `server.py`. The `server.py` file contains the following Python code:

```

1  from socket import *
2  import sys # In order to terminate the program
3  serverSocket = socket(AF_INET, SOCK_STREAM)
4  serverPort = 6789 # Use port 6789 as in the example
5  serverSocket.bind(('', serverPort)) # Bind to all available interfaces on this port
6  serverSocket.listen(1) # Start listening, with a backlog of 1
7  while True:
8      print('Ready to serve...')
9      connectionSocket, addr = serverSocket.accept() # Accept an incoming connection
10     try:
11         message = connectionSocket.recv(1024).decode() # Receive the HTTP request (up to 1024 bytes)
12         print(message) # Print the received message for debugging
13         filename = message.split()[1] # Parse the filename from the request (e.g., /HelloWorld.html)
14         f = open(filename[1:]) # Open the file, stripping the leading '/'
15         outputdata = f.read() # Read the entire content of the file
16         connectionSocket.send('HTTP/1.1 200 OK\r\n\r\n'.encode()) # Send HTTP header line into socket
17         for i in range(0, len(outputdata)):
18             connectionSocket.send(outputdata[i].encode())
19         connectionSocket.send("\r\n".encode())
20         connectionSocket.close()
21     except IOError:
22         response = 'HTTP/1.1 404 Not Found\r\n\r\n<html><body><h1>404 Not Found</h1></body></html>'
23         connectionSocket.send(response.encode())
24         connectionSocket.close()
25 serverSocket.close()
26 sys.exit() # Terminate the program after sending the corresponding data

```

The terminal at the bottom shows the following commands and output:

```

(base) alireza@alirezas-MacBook-Air 2 % kill python
(base) alireza@alirezas-MacBook-Air 2 % python server.py
Ready to serve...

```

شکل ۱: completed code

The screenshot shows a terminal with the following commands and output:

```

(base) alireza@alirezas-MacBook-Air 2 % ifconfig en0
en0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
    options=400<CHANNEL_IO>
    ether 80:65:7c:ea:db:b0
    inet6 fe80::19:9da6:8ee6:ea86%en0 prefixlen 64 secured scopeid 0xb
    inet 172.17.70.236 netmask 0xfffffe00 broadcast 172.17.71.255
    nd6 options=201<PERFORMNUD,DAD>
    media: autoselect
    status: active
(base) alireza@alirezas-MacBook-Air 2 % ipconfig getifaddr en0
172.17.70.236
(base) alireza@alirezas-MacBook-Air 2 %

```

شکل ۲: command line codes to retrieve IP

The screenshot shows a web browser window displaying a 'Hello World' page. The page content is as follows:

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Hello World Page</title>
5 </head>
6 <body>
7   <h1>Hello, World!</h1>
8   <p>If you can see this, your Python web server is working correctly.</p>
9 </body>
10 </html>

```

The terminal window shows the following commands and output:

```

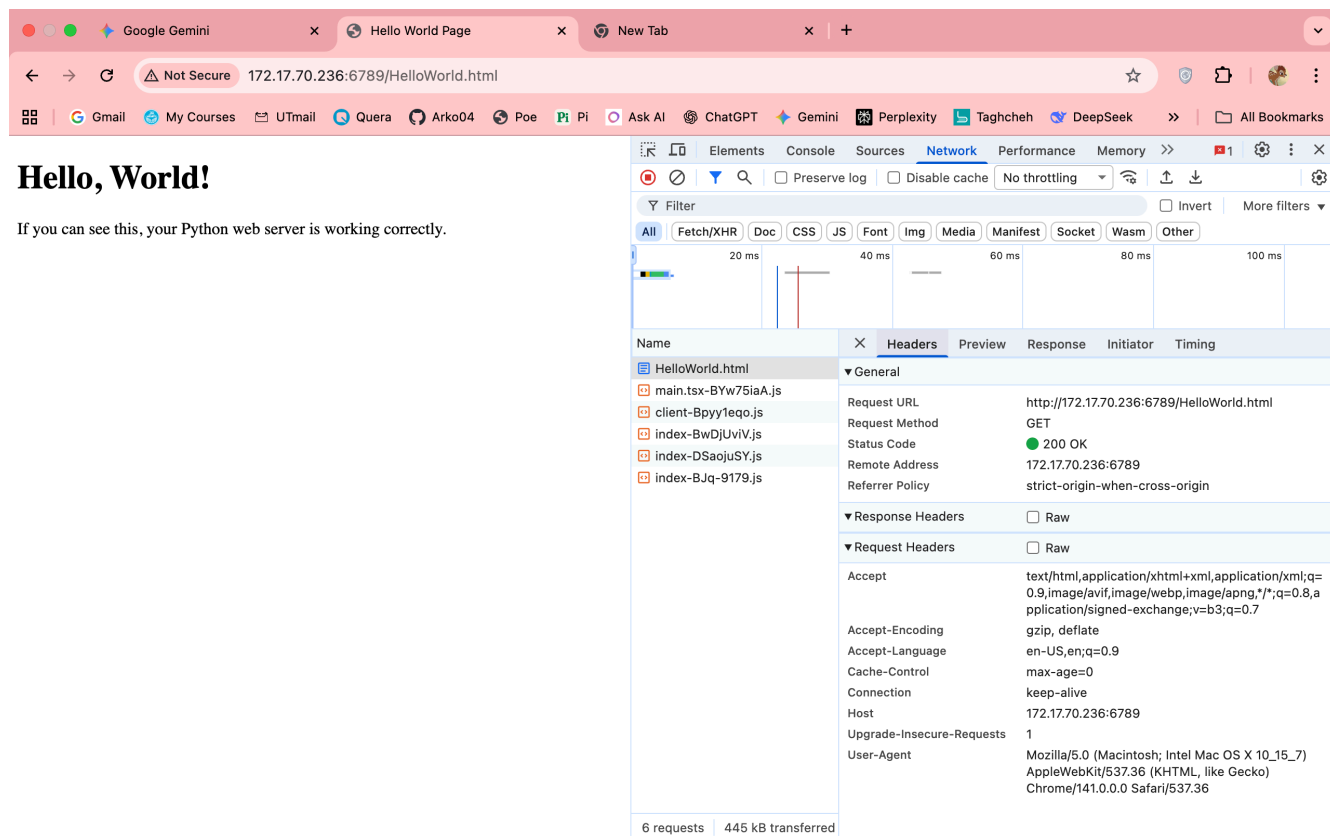
(base) alireza@alirezas-MacBook-Air 2 % clear
(base) alireza@alirezas-MacBook-Air 2 % pkill python
(base) alireza@alirezas-MacBook-Air 2 % python server.py
Ready to serve...
GET /HelloWorld.html HTTP/1.1
Host: 172.17.70.236:6789
Connection: keep-alive
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9

Ready to serve...
GET /FileNotFound.html HTTP/1.1
Host: 172.17.70.236:6789
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9

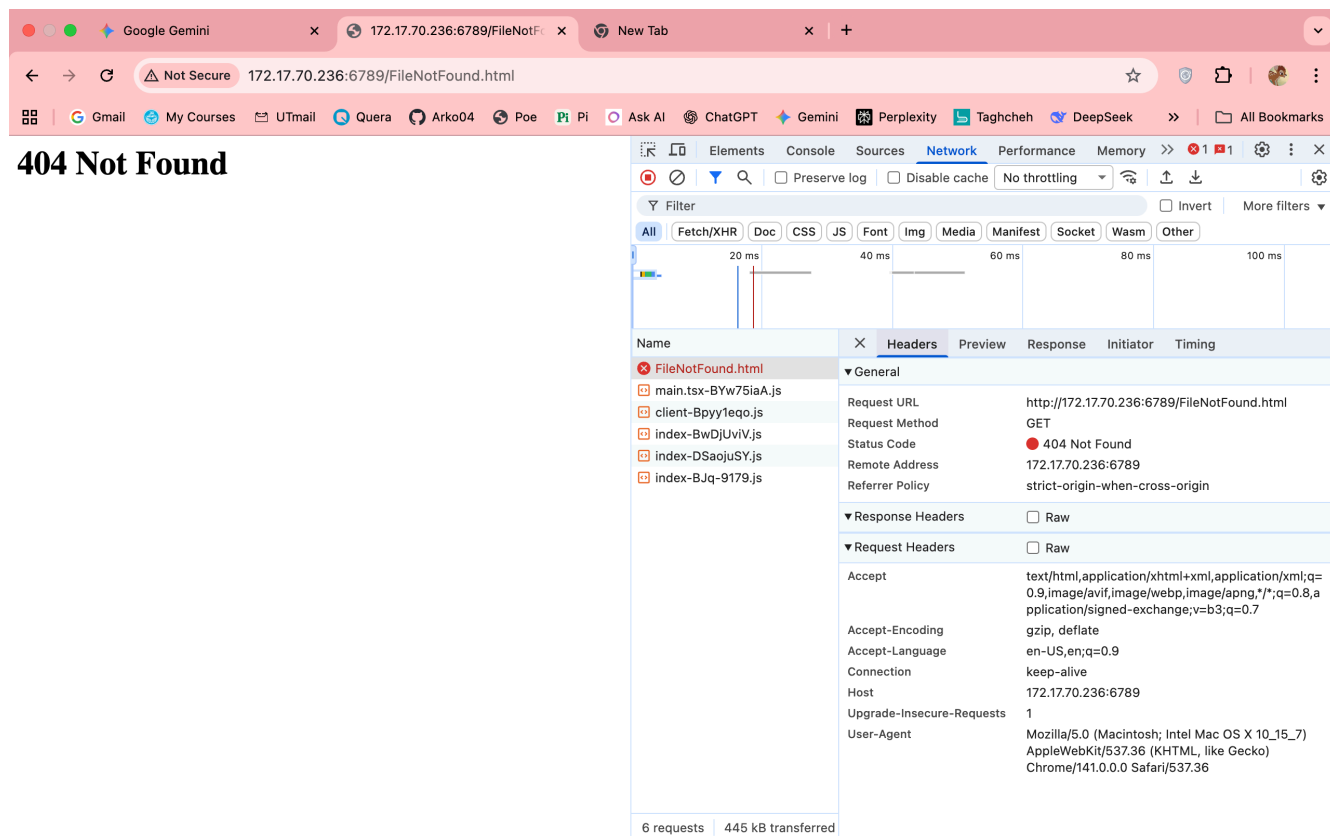
Ready to serve...

```

شکل ۳: content of HelloWorld.html file



شکل ۴: requesting HelloWorld.html file



شکل ۵: requesting a non-existent file

اگر اون 6789 : را در *URL* نمیاوردیم، و در کد سرور هم ست می‌کردیم که $serverPort = 80$ ، باز میدیدیم که میتوان محتوای همان فایل HelloWorld.html را مثل قبل دید. تست کردم شخصا. و خروجیش مثل ۴ شد.

تمرین اختیاری

بخش اول

در فایل thread.py کد مربوطه قرار گرفته است. عکس آن را میتوانید در ۶ ببینید. خروجی آن نیز در ?? قابل مشاهده است،

بخش دوم

کد مربوط به کلاینت را در client.py آوردم و در این عکس ۸ هم میتوانید آن را ببینید. و همانطور که در *stdout* مربوط به کلاینت میتوانید ببینید ۹، پاسخ سرور به کلاینت پس از یک درخواست موفق و یک درخواست ناموفق پرینت شده است.

مراجع

- [1] “Content-length-in-byte,” <https://gemini.google.com/share/4611da89965b>, 2025.
- [2] “Cashe-hit-in-lan,” <https://gemini.google.com/share/4dbb89549935>, 2025.
- [3] “San-francisco-to-new-york,” <https://gemini.google.com/share/68f261e652cb>, 2025.

```

thread copy.py > ...
1  from socket import *
2  import sys # In order to terminate the program
3  import threading # Import the threading module
4  def handle_client(connectionSocket, addr):
5      print(f"[Thread for {addr}] Connection started.")
6      try:
7          message = connectionSocket.recv(1024).decode()
8          if not message:
9              print(f"[Thread for {addr}] Empty request, closing connection.")
10             connectionSocket.close()
11             return
12
13             print(f"[Thread for {addr}] Received request: {message.splitlines()[0]}")
14             filename = message.split()[1]
15             f = open(filename[1:])
16             outputdata = f.read()
17             f.close() # Close the file
18             connectionSocket.send('HTTP/1.1 200 OK\r\n\r\n'.encode())
19             for i in range(0, len(outputdata)):
20                 connectionSocket.send(outputdata[i].encode())
21             connectionSocket.send("\r\n".encode())
22             print(f"[Thread for {addr}] Sent file: {filename[1:]}")
23         except IOError:
24             print(f"[Thread for {addr}] Error: File not found: {filename[1:]}")
25             response = 'HTTP/1.1 404 Not Found\r\n\r\n<html><body><h1>404 Not Found</h1></body></html>'
26             connectionSocket.send(response.encode())
27         except Exception as e:
28             print(f"[Thread for {addr}] An error occurred: {e}")
29         finally:
30             connectionSocket.close()
31             print(f"[Thread for {addr}] Connection closed.")
32     serverSocket = socket(AF_INET, SOCK_STREAM)
33     serverSocket.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
34     serverPort = 6789
35     serverSocket.bind(('', serverPort))
36     serverSocket.listen(5) # Increase backlog to 5 for a multithreaded server
37     print(f"Main server listening on port {serverPort}...")
38     try:
39         while True:
40             print("\nMain thread: Ready to serve, waiting for connection...")
41             connectionSocket, addr = serverSocket.accept()
42             print(f"Main thread: Accepted connection from {addr}")
43             client_thread = threading.Thread(target=handle_client, args=(connectionSocket, addr))
44             client_thread.start()
45     except KeyboardInterrupt:
46         print("\nShutting down server due to KeyboardInterrupt (Ctrl+C)...")
47     except Exception as e:
48         print(f"An error occurred in the main thread: {e}")
49     finally:
50         serverSocket.close()
51         sys.exit() # Terminate the program
52

```

شکل ۶: thread.py code

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
○ (base) alireza@alirezas-MacBook-Air 2 % python thread.py
Main server listening on port 6789...

Main thread: Ready to serve, waiting for connection...
Main thread: Accepted connection from ('172.18.132.94', 56361)
[Thread for ('172.18.132.94', 56361)] Connection started.

Main thread: Ready to serve, waiting for connection...
[Thread for ('172.18.132.94', 56361)] Received request: GET /HelloWorld.html HTTP/1.1
[Thread for ('172.18.132.94', 56361)] Sent file: HelloWorld.html
[Thread for ('172.18.132.94', 56361)] Connection closed.
Main thread: Accepted connection from ('172.18.132.94', 57675)
[Thread for ('172.18.132.94', 57675)] Connection started.

Main thread: Ready to serve, waiting for connection...
[Thread for ('172.18.132.94', 57675)] Received request: GET /HelloWorld.html HTTP/1.1
[Thread for ('172.18.132.94', 57675)] Sent file: HelloWorld.html
[Thread for ('172.18.132.94', 57675)] Connection closed.
Main thread: Accepted connection from ('172.18.132.94', 57719)
[Thread for ('172.18.132.94', 57719)] Connection started.

Main thread: Ready to serve, waiting for connection...
[Thread for ('172.18.132.94', 57719)] Received request: GET /HelloWorld.html HTTP/1.1
[Thread for ('172.18.132.94', 57719)] Sent file: HelloWorld.html
[Thread for ('172.18.132.94', 57719)] Connection closed.
Main thread: Accepted connection from ('172.18.132.94', 57908)
[Thread for ('172.18.132.94', 57908)] Connection started.

Main thread: Ready to serve, waiting for connection...
[Thread for ('172.18.132.94', 57908)] Received request: GET /Hello.html HTTP/1.1
[Thread for ('172.18.132.94', 57908)] Error: File not found: Hello.html
[Thread for ('172.18.132.94', 57908)] Connection closed.
Main thread: Accepted connection from ('172.18.132.94', 57940)
[Thread for ('172.18.132.94', 57940)] Connection started.

Main thread: Ready to serve, waiting for connection...
[Thread for ('172.18.132.94', 57940)] Received request: GET /Hello.html HTTP/1.1
[Thread for ('172.18.132.94', 57940)] Error: File not found: Hello.html
[Thread for ('172.18.132.94', 57940)] Connection closed.
Main thread: Accepted connection from ('172.18.132.94', 58275)
[Thread for ('172.18.132.94', 58275)] Connection started.

Main thread: Ready to serve, waiting for connection...
[Thread for ('172.18.132.94', 58275)] Received request: GET /HelloWorld.html HTTP/1.1
[Thread for ('172.18.132.94', 58275)] Sent file: HelloWorld.html

```

شکل ۷: thread.py output

```

client copy.py > ...
1  from socket import *
2  import sys # For reading command-line arguments
3  if len(sys.argv) != 4:
4      print("Usage: python client.py server_host server_port filename")
5      sys.exit()
6  server_host = sys.argv[1]
7  try:
8      server_port = int(sys.argv[2])
9  except ValueError:
10     print(f"Error: server_port '{sys.argv[2]}' must be an integer.")
11     sys.exit()
12
13  filename = sys.argv[3]
14  try:
15      clientSocket = socket(AF_INET, SOCK_STREAM)
16      print(f"Socket created...")
17  except error as e:
18      print(f"Error creating socket: {e}")
19      sys.exit()
20  try:
21      print(f"Connecting to {server_host} on port {server_port}...")
22      clientSocket.connect((server_host, server_port))
23      print("Connection successful.")
24  except error as e:
25      print(f"Error connecting to server: {e}")
26      sys.exit()
27  request = f"GET /{filename} HTTP/1.1\r\nHost: {server_host}\r\n\r\n"
28
29  try:
30      clientSocket.send(request.encode())
31      print(f"Sent GET request for: /{filename}")
32  except error as e:
33      print(f"Error sending data: {e}")
34      clientSocket.close()
35      sys.exit()
36  print("\n--- Server Response ---")
37  response = ""
38  while True:
39      data = clientSocket.recv(1024)
40      if not data:
41          break
42      response_chunk = data.decode('utf-8', 'ignore')
43      print(response_chunk, end='') # Print the chunk without adding a new line
44  print("\n--- End of Response ---")
45  clientSocket.close()
46  print("Connection closed.")
47

```

شکل ۸: client code

The screenshot shows a VS Code editor with three files open: `HelloWorld.html`, `server.py`, and `client.py`. The `client.py` file is active, showing the following code:

```

20
21 # --- 1. Create a client socket ---
22 # AF_INET = Use IPv4
23 # SOCK_STREAM = Use TCP
24 try:
25     clientSocket = socket(AF_INET, SOCK_STREAM)
26     print(f"Socket created")

```

The terminal output shows the execution of the client script:

```

(base) alireza@alirezas-MacBook-Air 2 % python client.py 172.17.70.236 6789 HelloWorld.html
Socket created...
Connecting to 172.17.70.236 on port 6789...
Connection successful.
Sent GET request for: /HelloWorld.html

--- Server Response ---
HTTP/1.1 200 OK

<!DOCTYPE html>
<html>
<head>
  <title>Hello World Page</title>
</head>
<body>

  <h1>Hello, World!</h1>
  <p>If you can see this, your Python web server is working correctly.</p>

</body>
</html>

--- End of Response ---
Connection closed.
(base) alireza@alirezas-MacBook-Air 2 % python client.py 172.17.70.236 6789 FileNotFound.html
Socket created...
Connecting to 172.17.70.236 on port 6789...
Connection successful.
Sent GET request for: /FileNotFound.html

--- Server Response ---
HTTP/1.1 404 Not Found

<html><body><h1>404 Not Found</h1></body></html>
--- End of Response ---
Connection closed.
(base) alireza@alirezas-MacBook-Air 2 %

```

شکل ۹: after running the client.py file